Object Oriented Design

Inheritance

A **class** is a type that represents an entity or concept.

- A class has fields and methods.
- The methods have an implementation.

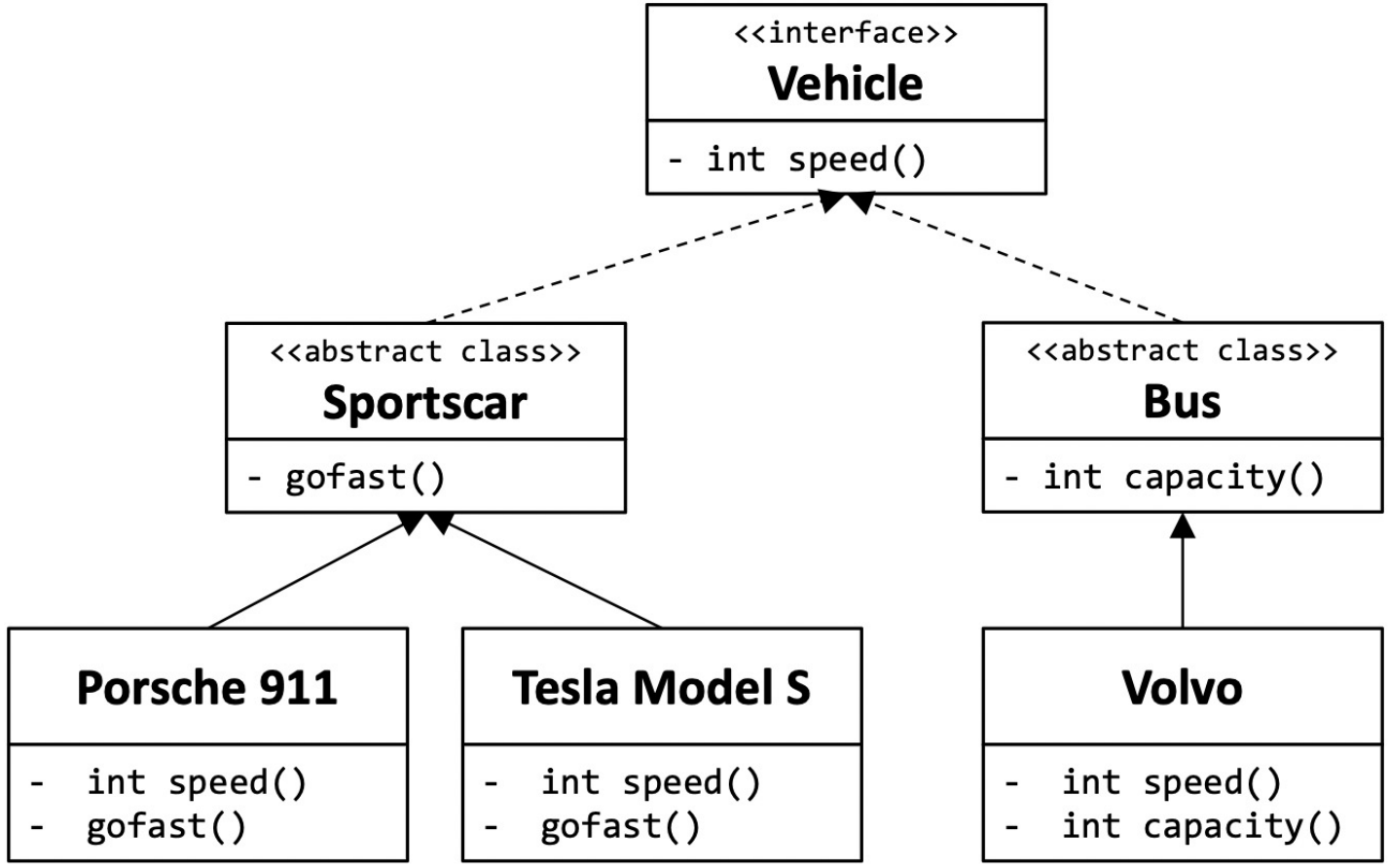
An **interface** is a type that represents an entity or concept.

- An interface has methods, but no fields.
- The methods have no implementation, only a method signature.

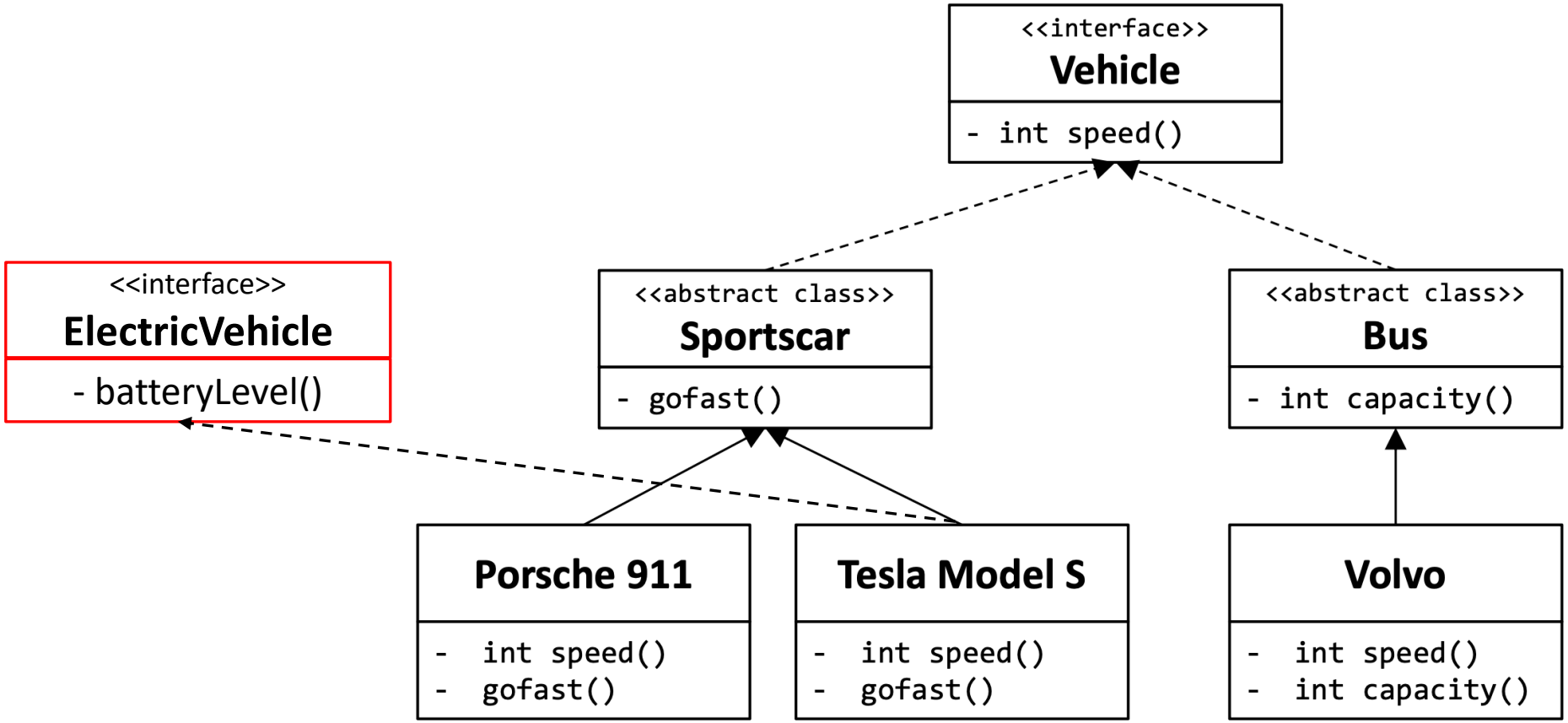
An **abstract class** is a type that represents an entity or concept.

- An abstract class has fields and methods.
- The methods may or may not have an implementation.
- A concept in-between an interface and a class.

Example of OO Design



Multiple Inheritance



What we can do

We have **a lot of flexibility** when creating **class hierarchies**, but:

- We are not allowed to create cycles in the inheritance hierarchy.
- We are not allowed to let an interface extend a class.
- We cannot extend an interface or class marked **final**.
- We cannot override a method marked **final**.

What we should not do

We can use inheritance to create complex hierarchies of interfaces, abstract classes, and classes.

But because we can, does not mean that we should.

Here is a bit of advice:

- Use interfaces.
- Use classes that implement interfaces.
- Avoid deep inheritance.
- Avoid abstract classes.